

Deep Learning for Image Analysis

Mehyar MLAWEH



Master BDIA - Dauphine Tunis

PhD - Université PSL



Generative AI Engineer - Wealthy Technology

- ❑ Final Project: **70%**
- ❑ Lab Session Reports: **20%**
- ❑ Quizzes: **10%**

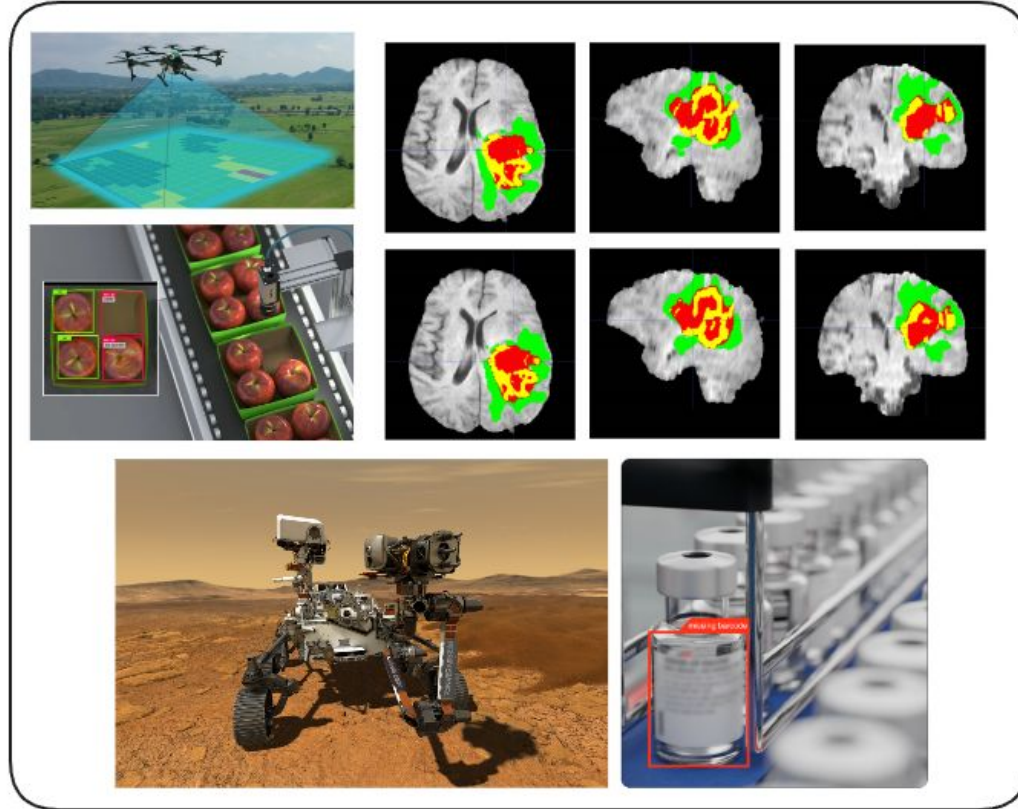
1. Computer Vision Foundations
2. CNNs
3. Transfer Learning
4. Vision Transformers
5. Object Detection
6. Segment Anything Model (SAM)
7. Final Project

Lecture 1:

Computer Vision Foundations

At an **abstract** level, the goal of computer vision problems is to use the observed image data to infer something about the world.

- Page 83, Computer Vision: Models, Learning, and Inference, 2012



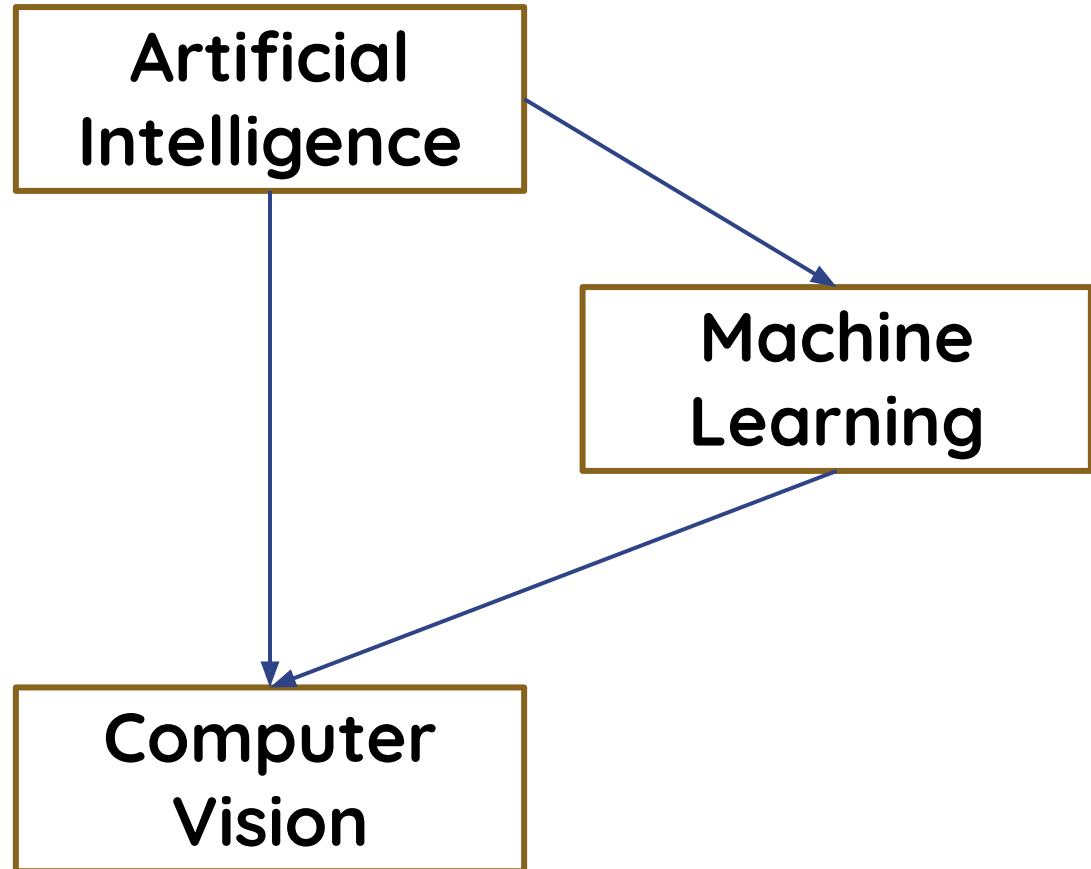


Image Classification

What is in the image?



Object Detection

What objects are present and where are they?



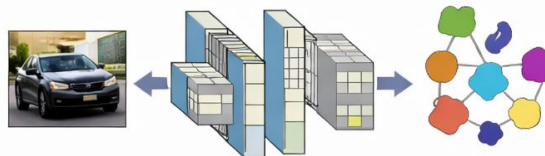
Image Segmentation

Which pixels belong to which object?

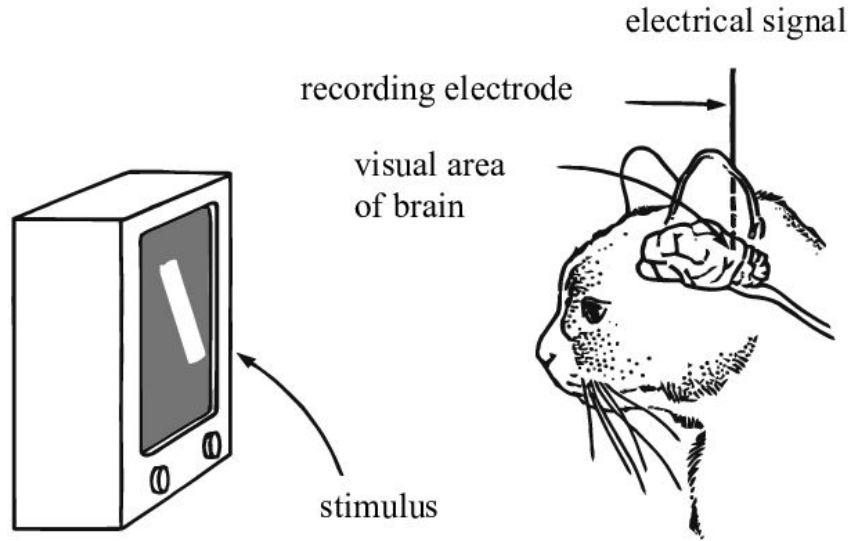


Representation Learning

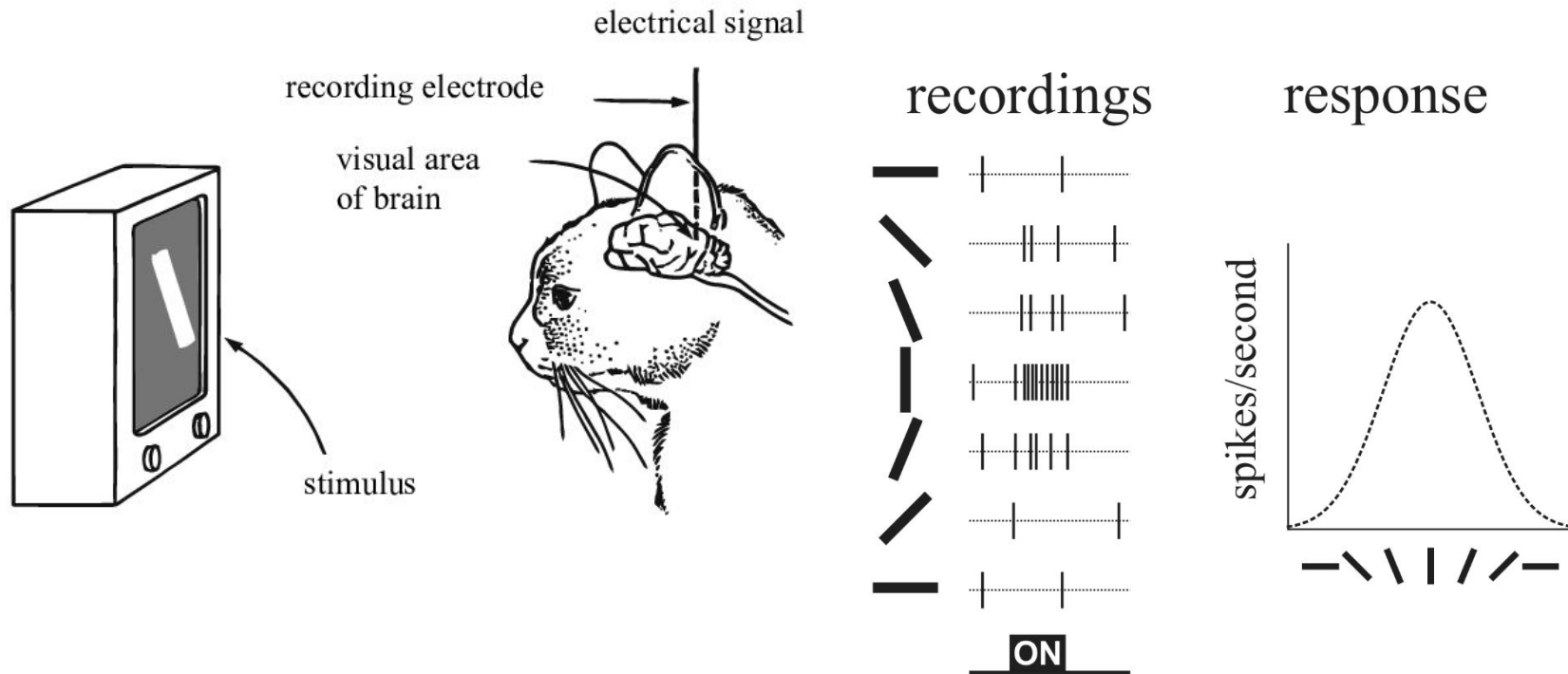
Learning visual features automatically?

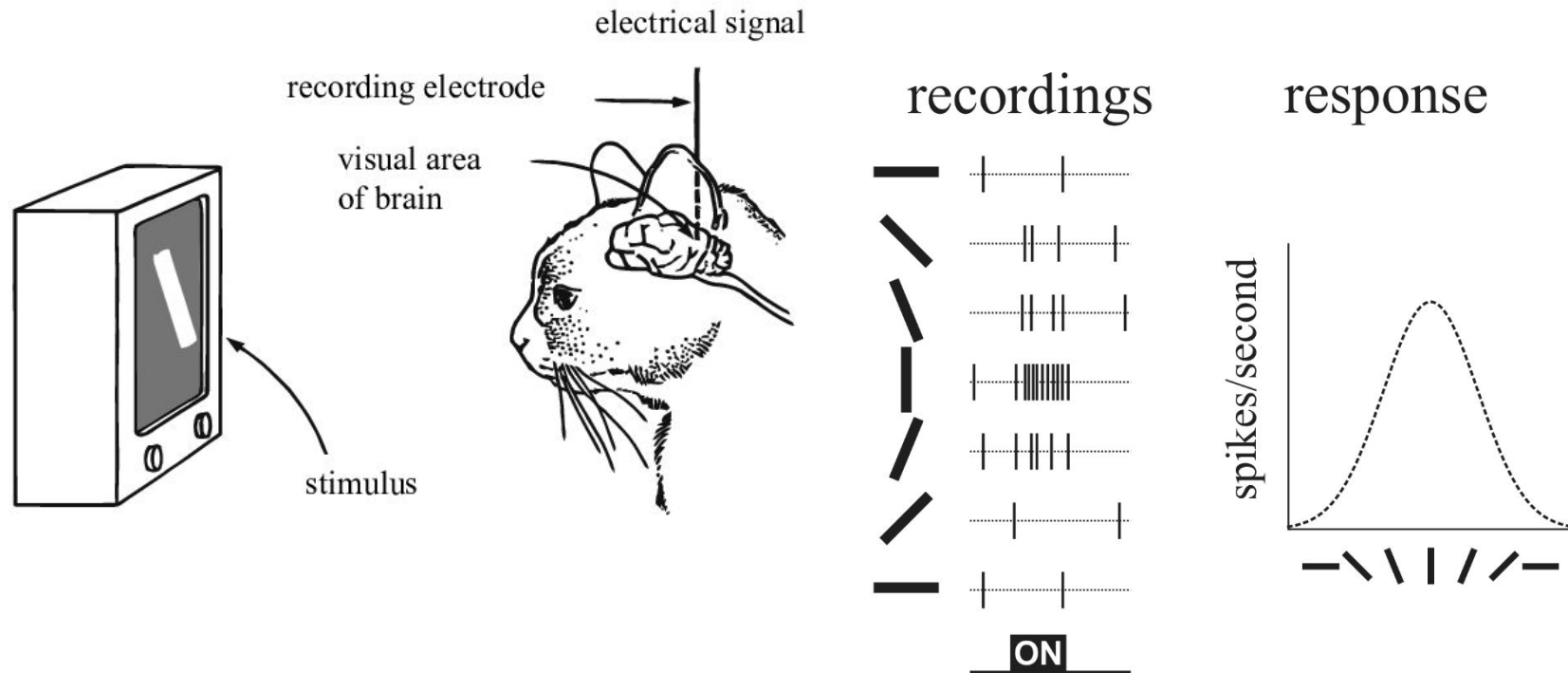


Hubel & Wiesel (1959)



Hubel & Wiesel (1959)

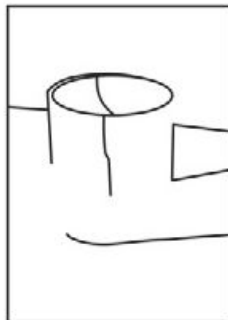




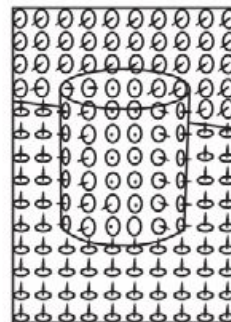
This discovery introduced a key idea: **vision is hierarchical**



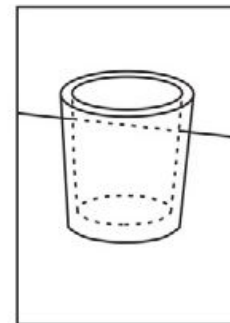
Retinal Image



Primal
Sketch



2.5D
Sketch



3D
Sketch

This framework strongly influenced **computer vision** research for decades

Edges, Parts, and Features

(1980s–2000s)

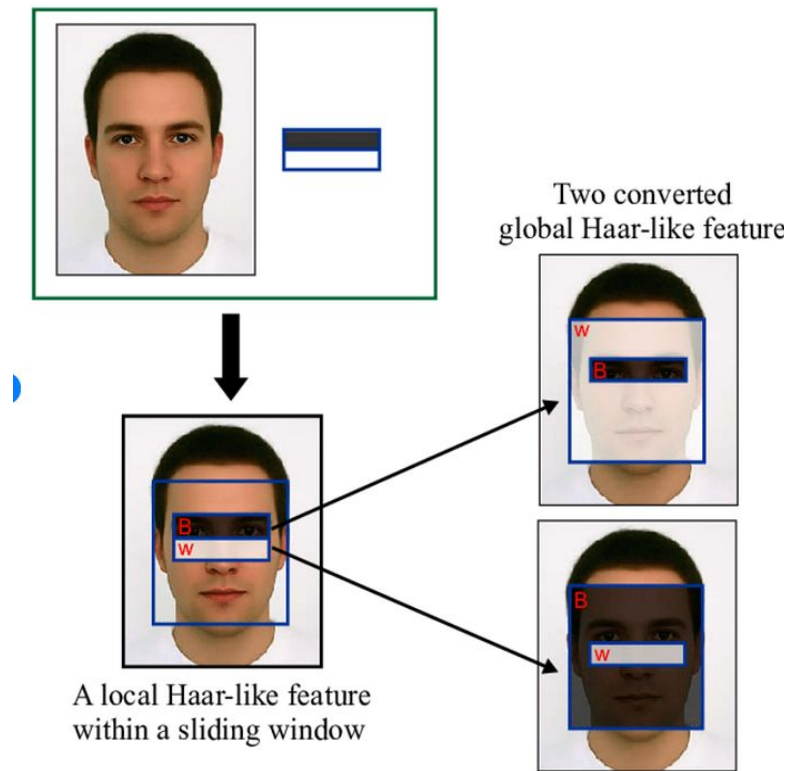
Researchers attempted to recognize objects using:

- ❑ Geometric primitives (generalized cylinders)
- ❑ Edges and contours

Notable milestones:

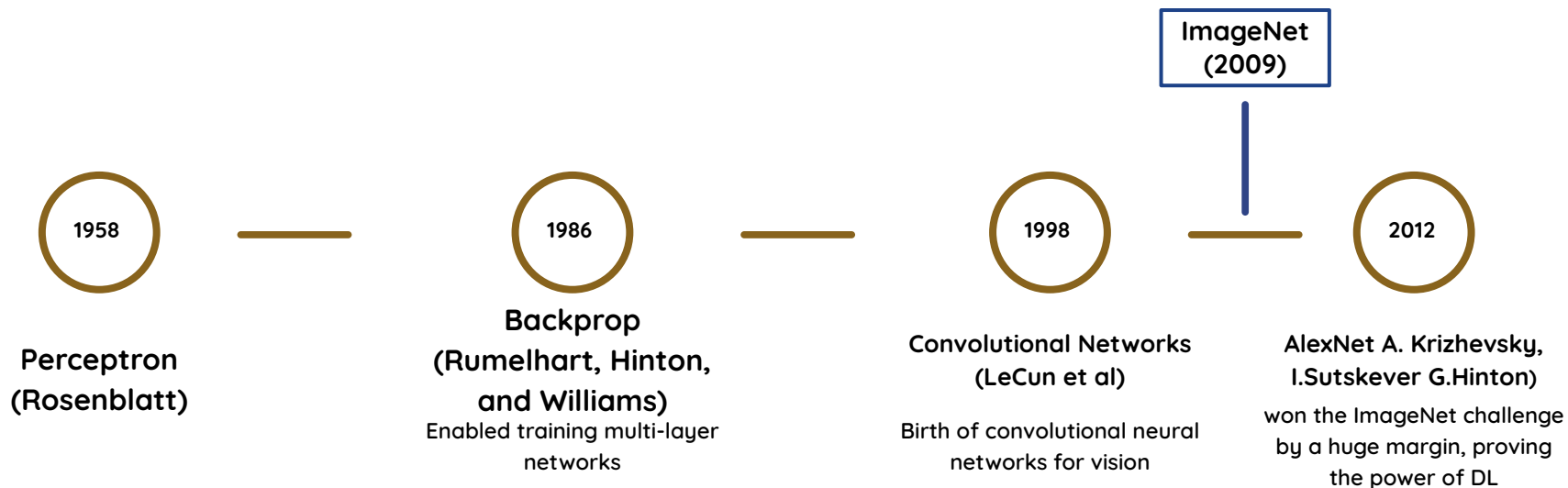
- ❑ Generalized Cylinders (Brooks & Binford, 1979)
- ❑ Canny Edge Detector (1986), one of the most influential algorithms in vision

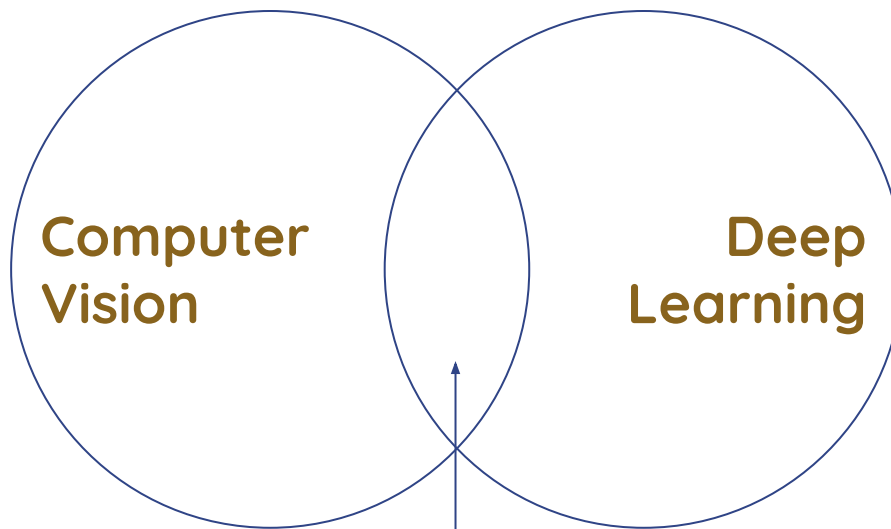
These methods were elegant and mathematically grounded, but **sensitive to noise** and **fragile in real-world images**.



It relies on fixed Haar-like features rather than **learning** from data.

The Birth of Neural Networks (1950s–1980s)

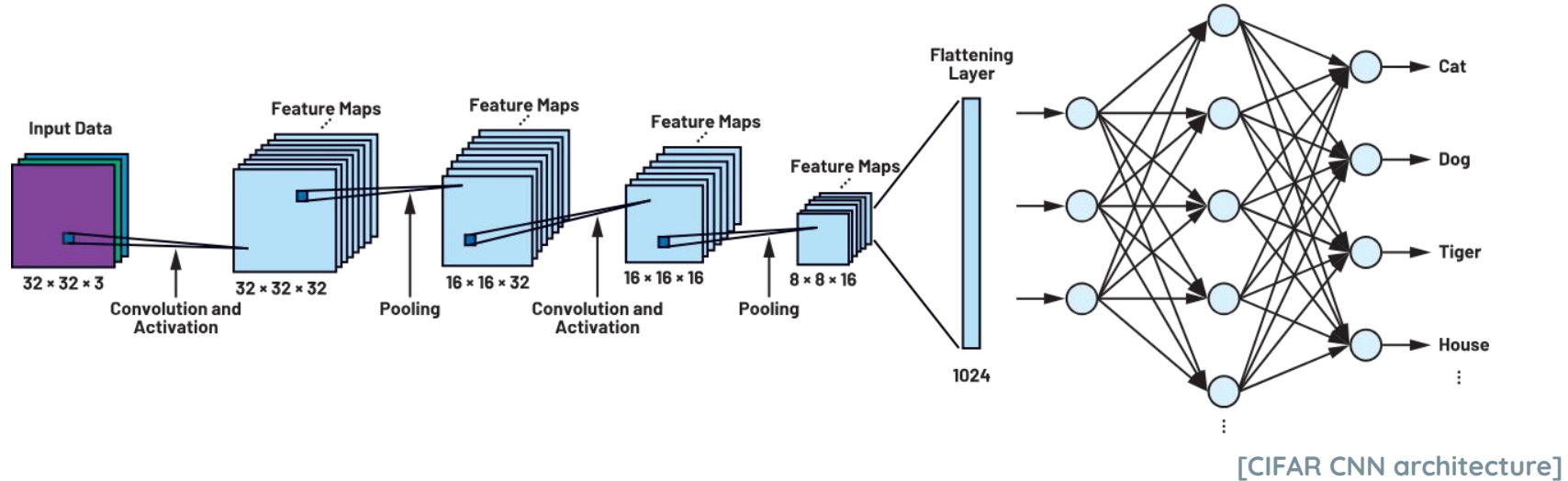




CNNs are the point where
rule-based vision ideas
became **learnable**.

After 2010 :

- **Large-scale data (ImageNet, 2009)**
- **GPUs & computation**
- **Deeper, trainable architectures**



📌 CNNs are rarely used alone today, they act as **backbones**, reused across tasks.

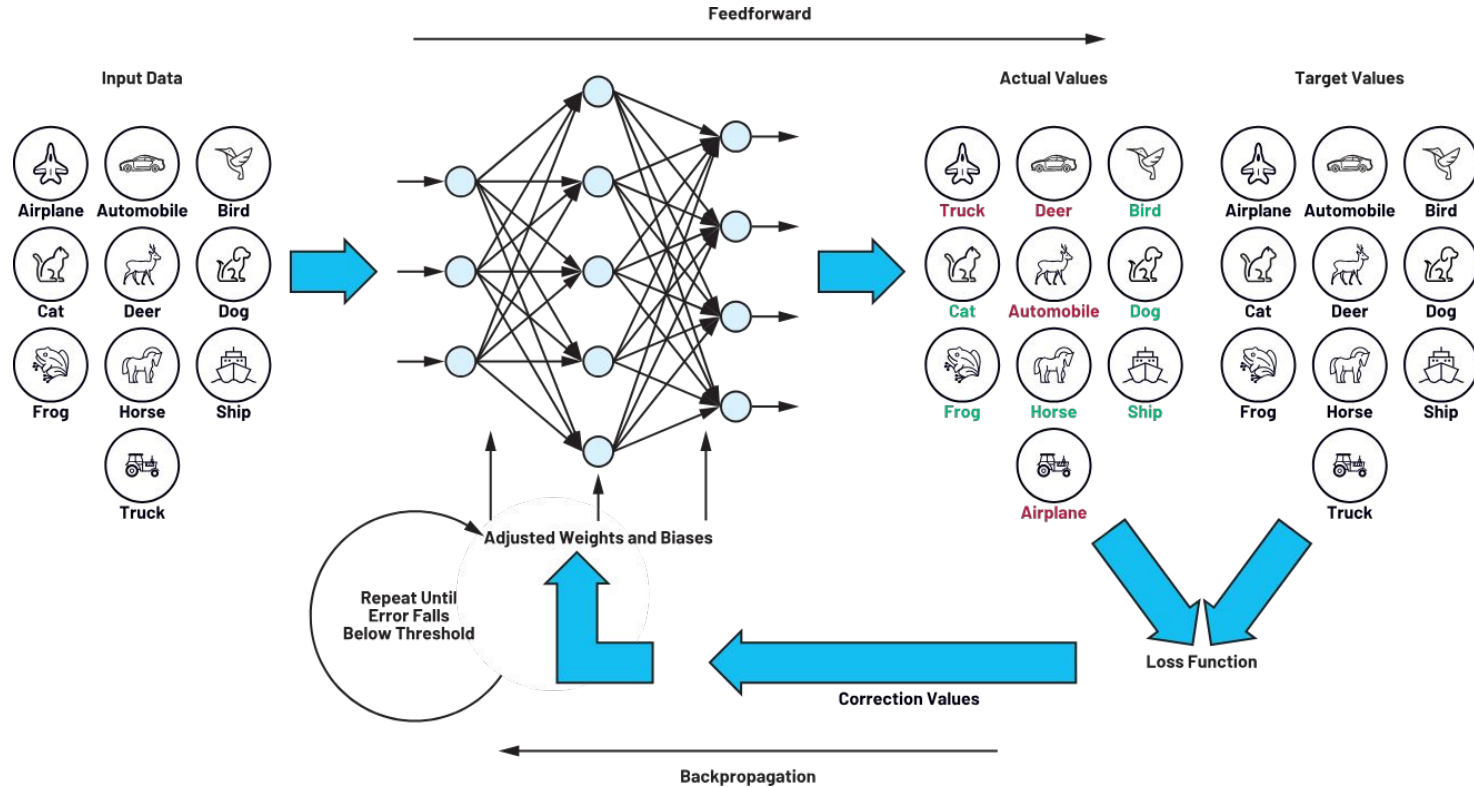


What Are Convolutional Neural Networks (CNNs)?

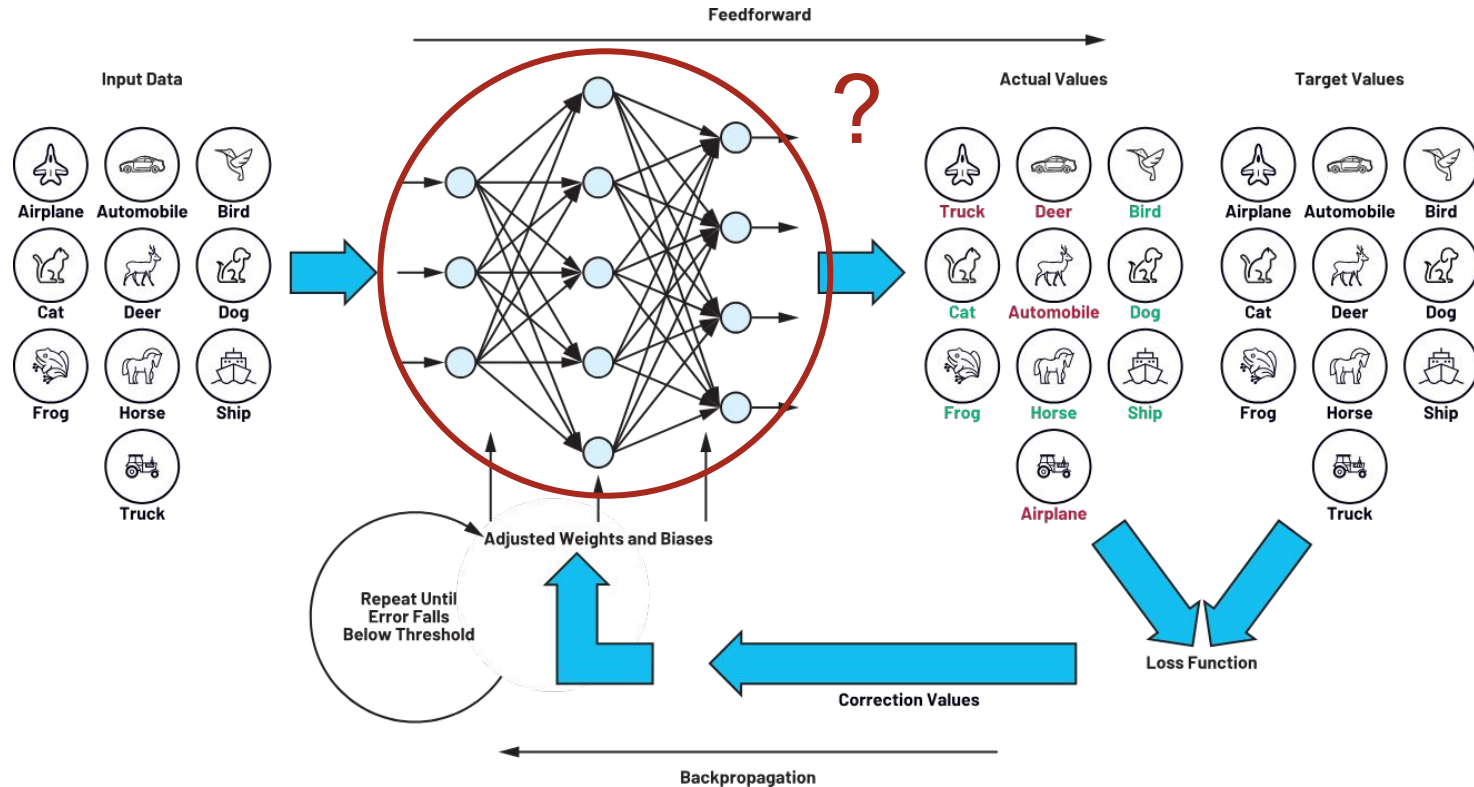
- **Convolutional Layers**
Learn local patterns (edges, textures, shapes) using learnable filters.
- **Activation Functions (e.g., ReLU)**
Introduce non-linearity to model complex visual patterns.
- **Pooling Layers**
Reduce spatial dimensions and improve robustness to small variations.
- **Fully Connected Layers**
Combine learned features to make final predictions.

How Are CNNs Trained ?

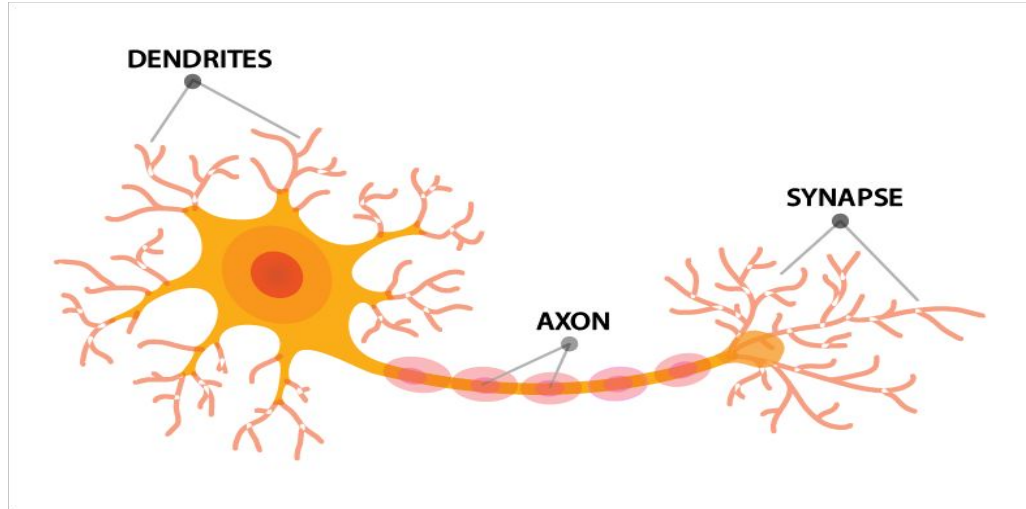
Supervised learning approach

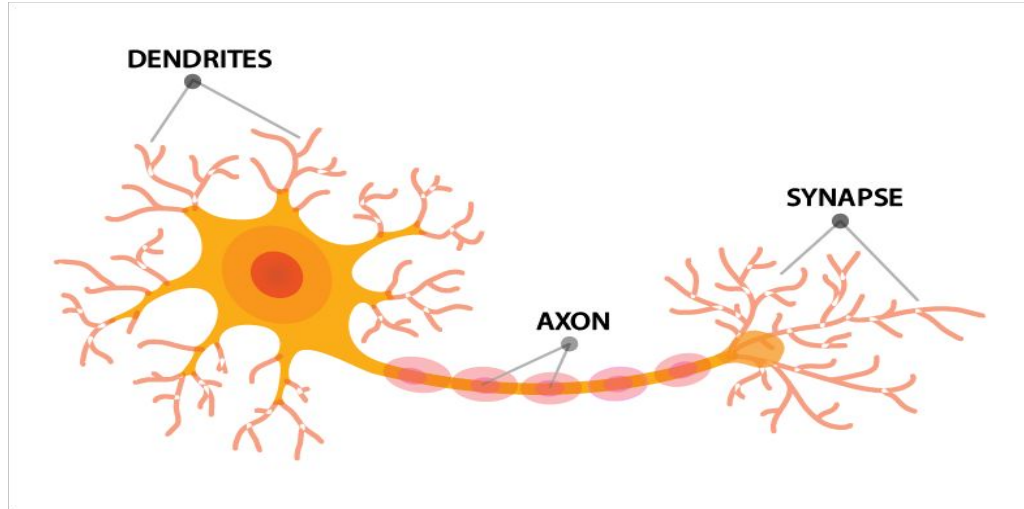


Supervised learning approach

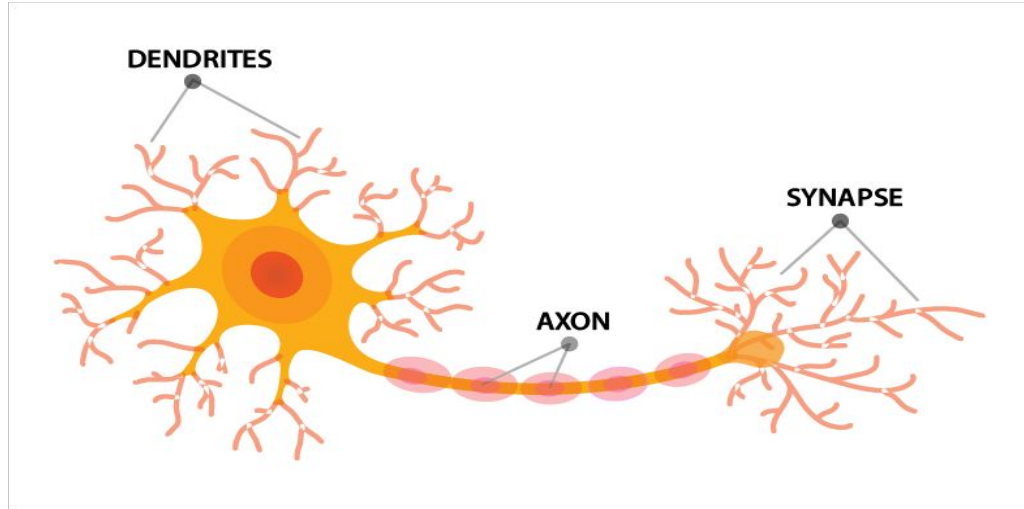


Backup

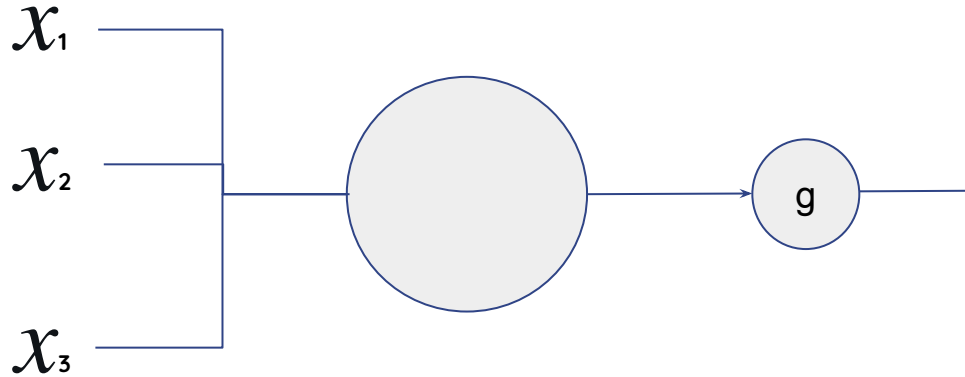




- A human neuron can have several thousand **dendrites**.

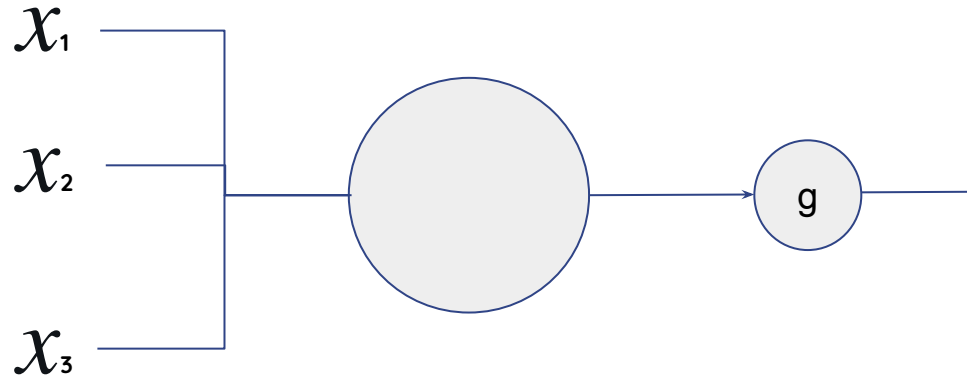


- A human neuron can have several thousand **dendrites**.
- The neuron sends a signal through its axon if during a given interval of time the net input signal is larger than a threshold.

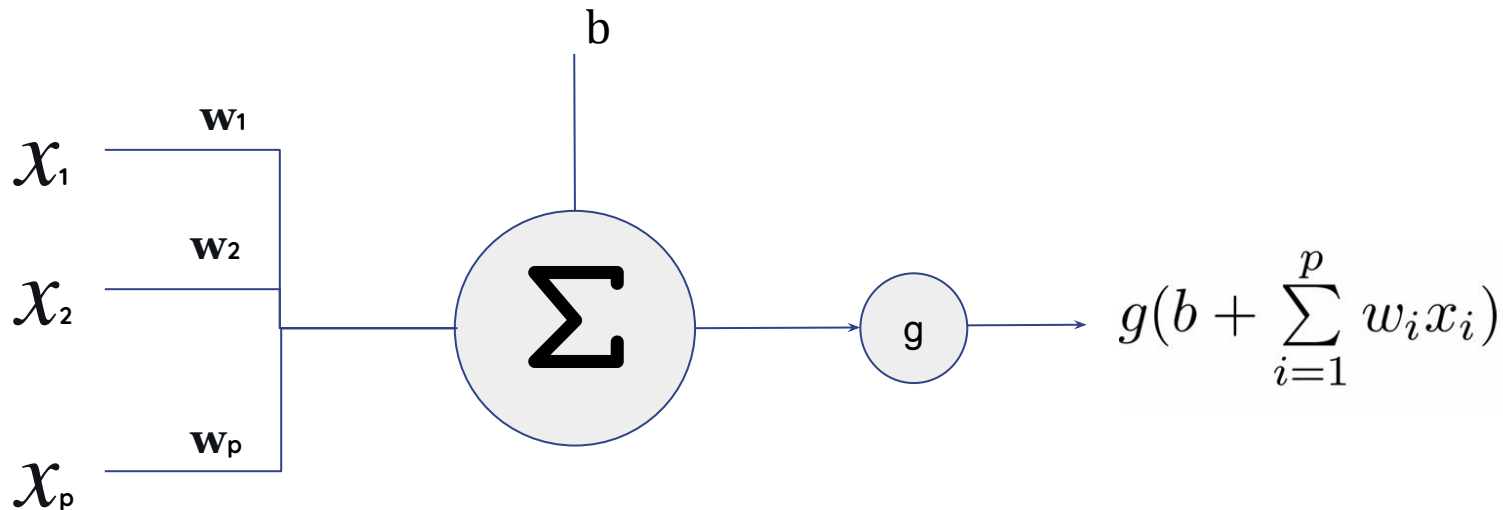


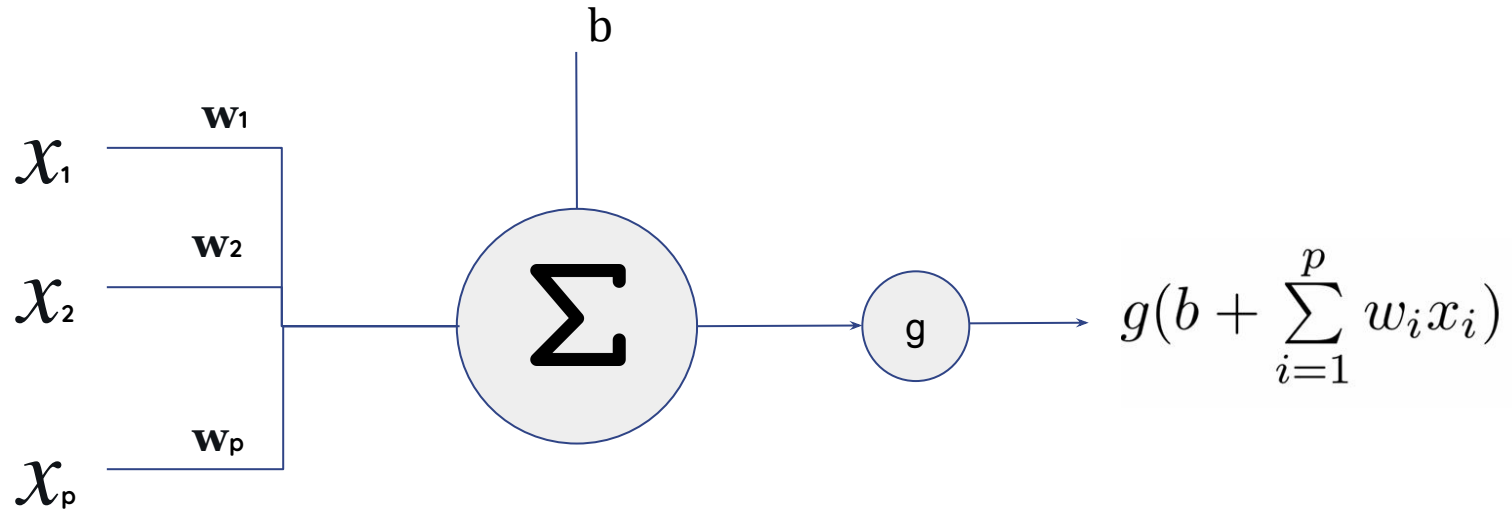
General principle

An artificial neuron takes p inputs $\{x_1, x_2, \dots, x_p\}$, combines them into a single value, and applies an **activation function** $g(\cdot)$ to produce the output.



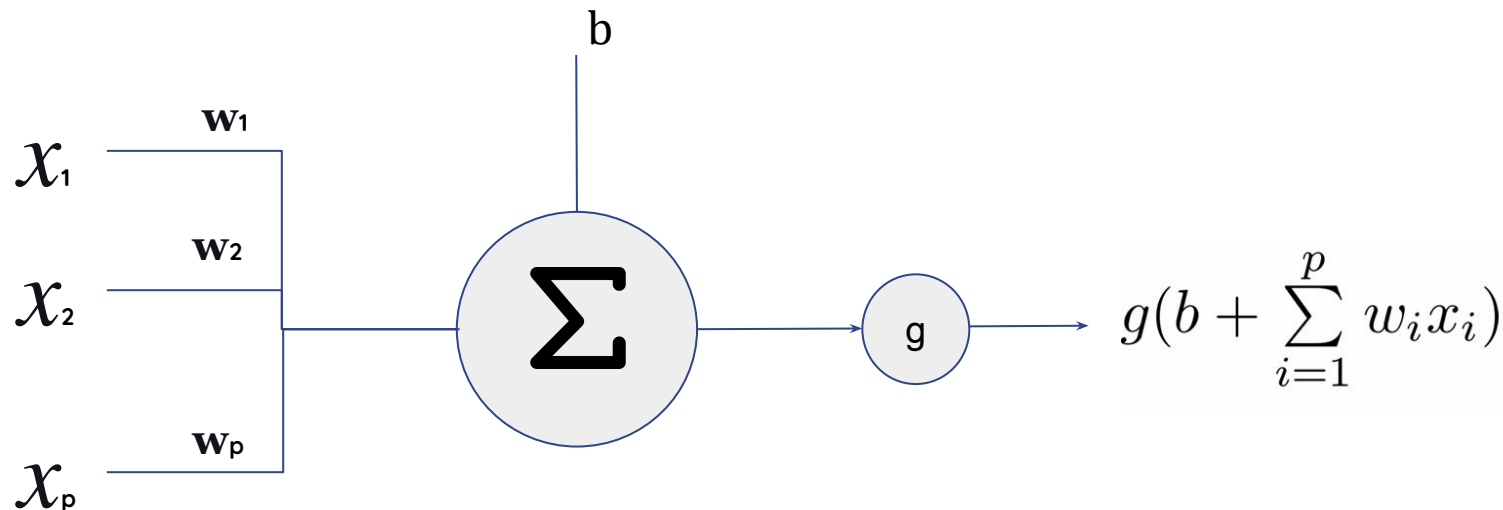
- The first artificial neuron model was proposed by [McCulloch and Pitts, 1943]
- Input and output signals were binary





The neuron computes a linear combination of the **inputs** x_i

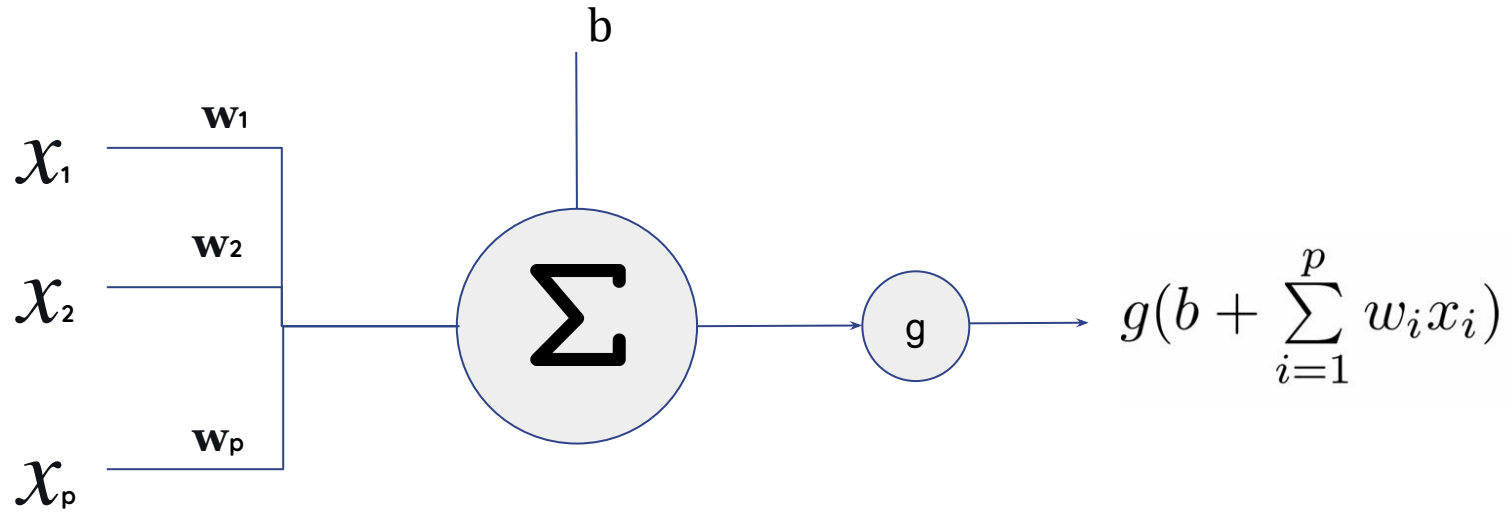
- The **weights** w_i are multiplied with the inputs
- The **bias** b can be interpreted as a threshold on the sum



The neuron computes a linear combination of the **inputs** x_i

- The **weights** w_i are multiplied with the inputs
- The **bias** b can be interpreted as a threshold on the sum

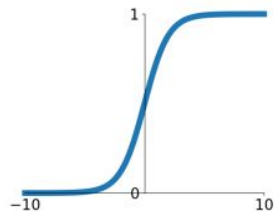
The **activation function** g somehow decides, depending on its input, if a signal (the neuron's activation) is produced.



- It's like a **gate**
- If its input is “**high enough**”, then the neuron is activated, i.e. a signal (other than zero) is produced
- It can be interpreted as a source of abstraction: information considered as unimportant is **ignored** (or **reduced**)

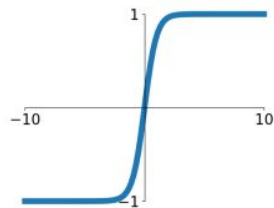
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



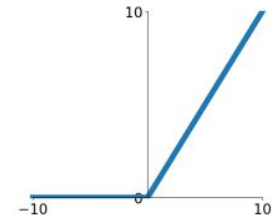
tanh

$$\tanh(x)$$



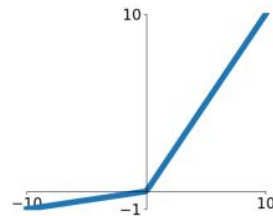
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

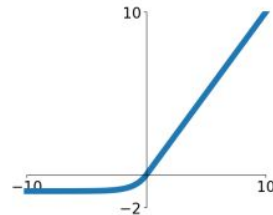


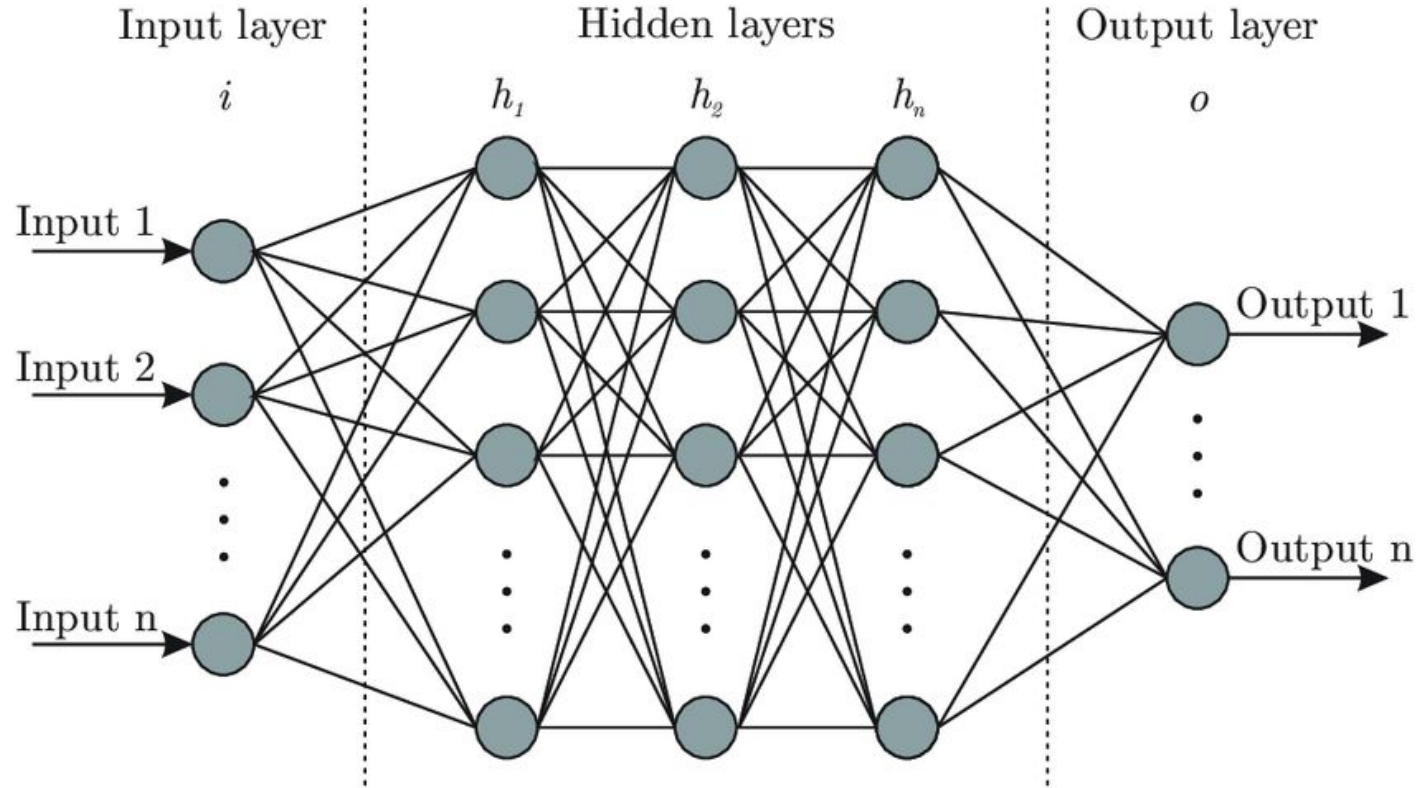
Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

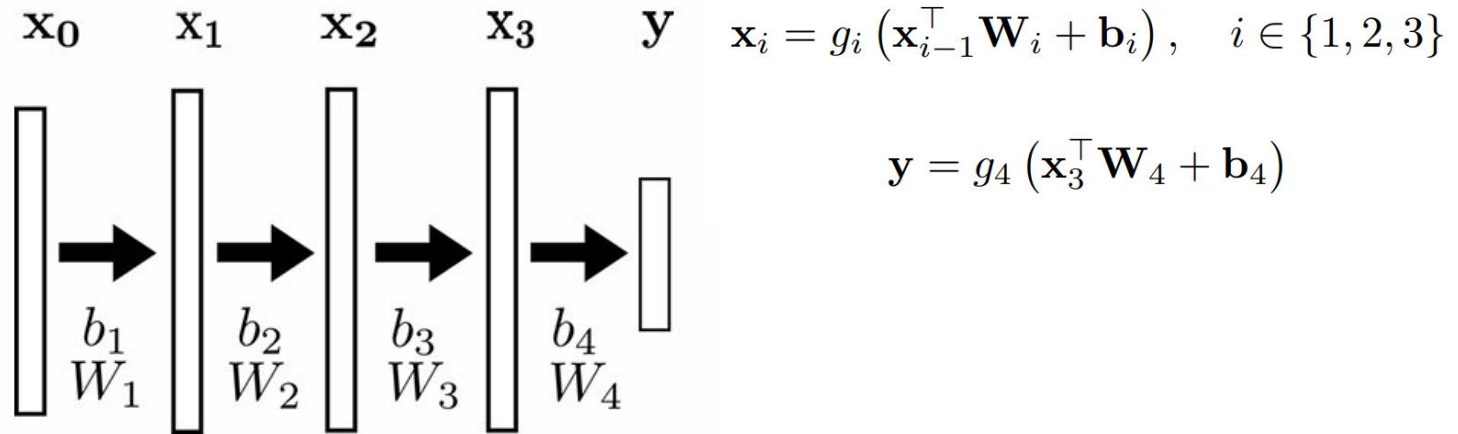




Fully-connected layer :

- A layer is said to be fully-connected (FC) if each of its neurons is connected to all the neurons of the previous layer

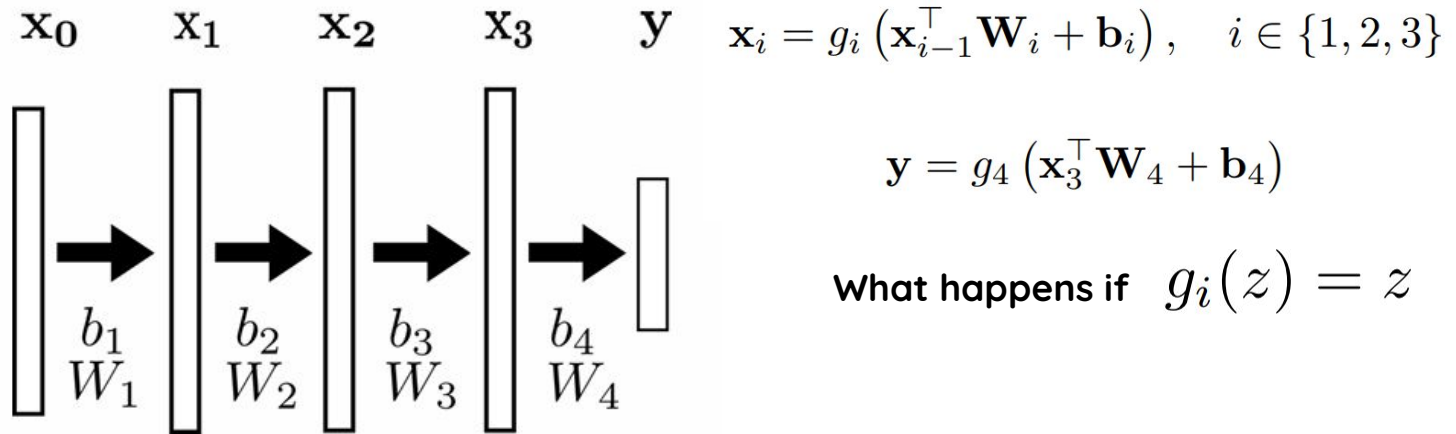
The equation of a FC neural network :



Fully-connected layer :

- A layer is said to be fully-connected (FC) if each of its neurons is connected to all the neurons of the previous layer

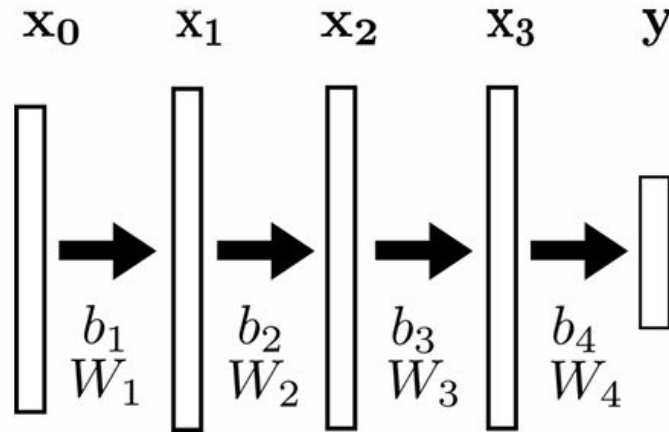
The equation of a FC neural network :



Fully-connected layer :

- A layer is said to be fully-connected (**FC**) if each of its neurons is connected to all the neurons of the previous layer

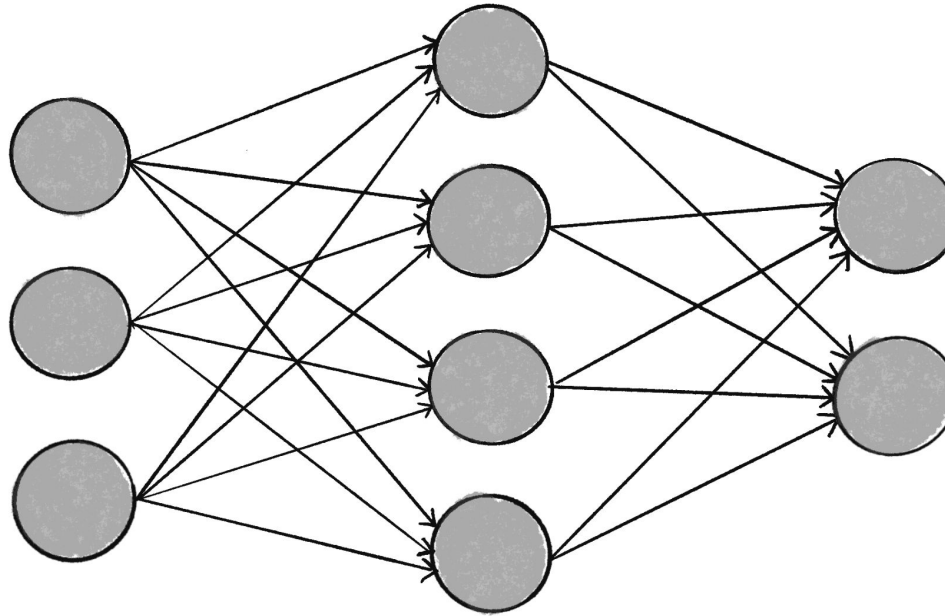
The equation of a FC neural network :



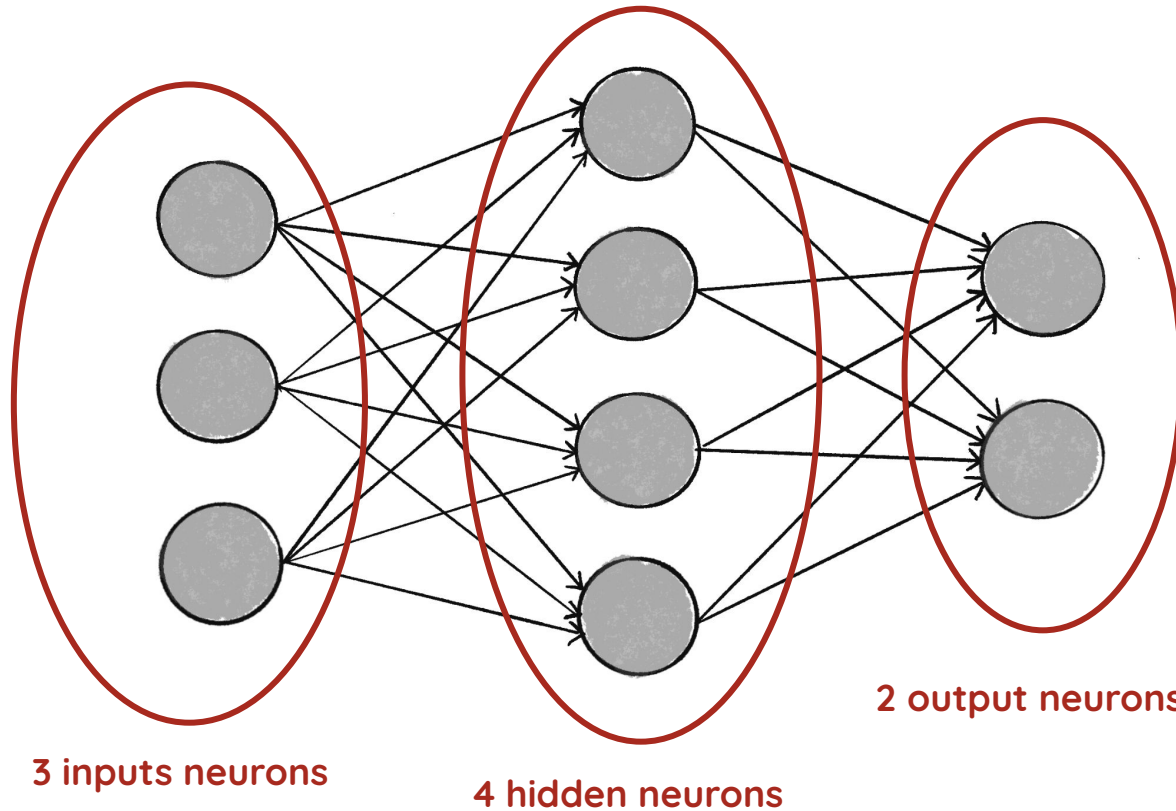
What happens if $g_i(z) = z$

Without **nonlinear activations**, a deep FC neural network is mathematically equivalent to a single linear (affine) layer

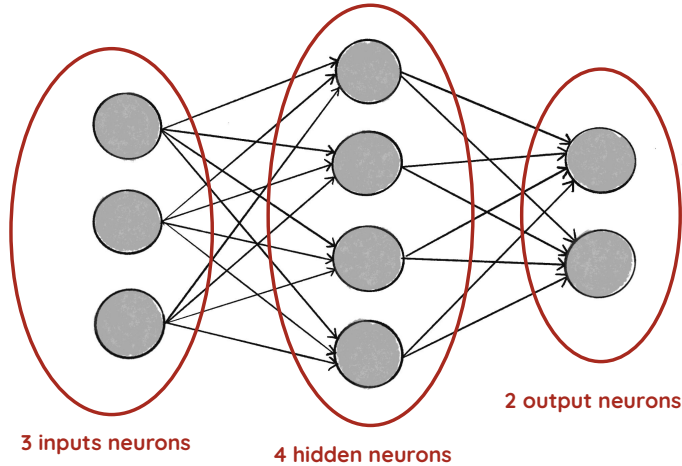
Number of parameters ?



Number of parameters ?



Number of parameters ?



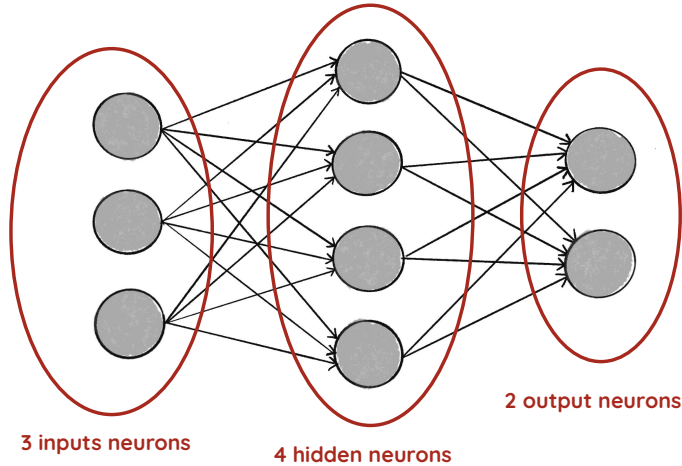
Rule to remember !

For two connected layers:

$$\text{Parameters} = (\text{inputs} \times \text{neurons}) + (\text{neurons})$$

Weights biases

Number of parameters ?



Rule to remember !

For two connected layers:

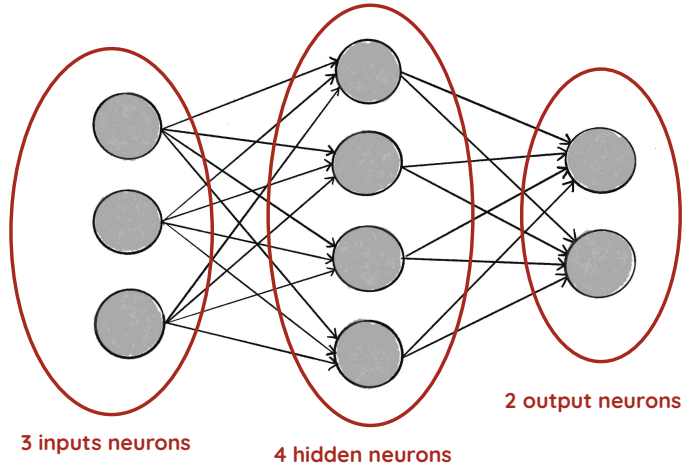
$$\text{Parameters} = (\text{inputs} \times \text{neurons}) + (\text{neurons})$$

Weights biases

So what's the number of parameters ?

- A/ 20
- B/ 26
- C/ 16

Number of parameters ?



$$(3 \times 4) + 4 = 16$$

$$(4 \times 2) + 2 = 10$$

$$10 + 16 = 26$$

Rule to remember !

For two connected layers:

$$\text{Parameters} = (\text{inputs} \times \text{neurons}) + (\text{neurons})$$

Weights

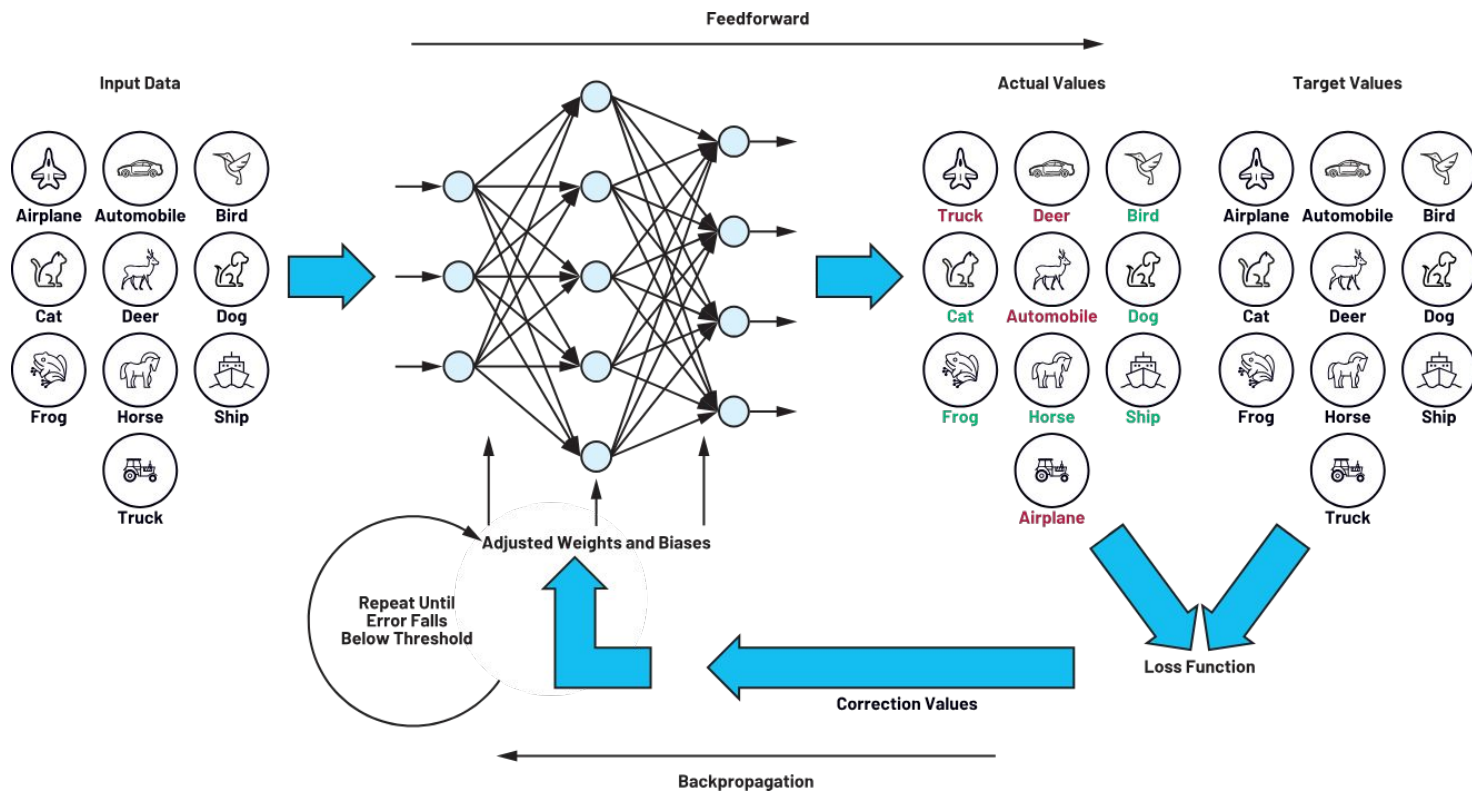
biases

So what's the number of parameters ?

A/ 20

B/ 26

C/ 16



What is a loss function:

A loss that measures how wrong the network's prediction is.

- Mean Squared Error (MSE)
- Cross-Entropy Loss : used in image classifications problems

Training is an **optimization problem**, where we want to find the weights that minimize the **loss function**

The problem that :

- Networks have millions of parameters
- Each parameter affects the loss indirectly
- We need an efficient way to compute influence

What is a Gradient?

- During training, we want to minimize the loss
- The gradient tells us:
How much the **loss changes** when a **parameter changes**

- For a weight w : $\frac{\partial t}{\partial w}$

If I slightly change w , how does the loss change?

Why Do We Need Gradients?

- Neural networks have millions of parameters
- We don't know the **best weights** directly
- Gradients tell us:
 - Which **direction** reduces the loss
 - How strongly each parameter influences the loss

Gradient Descent:

Parameters of the neural network are updated iteratively

$$\begin{aligned} &\text{Repeat until converge } \{ \\ &\quad w = w - \alpha \left[\frac{\partial Loss}{\partial w} \right] \\ &\quad b = b - \alpha \left[\frac{\partial Loss}{\partial b} \right] \\ &\} \end{aligned}$$

Gradient Descent:

Parameters of the neural network are updated iteratively

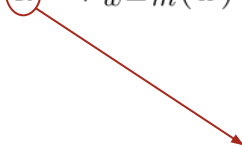
Repeat Until Convergence {

for i = 1...m {

$$\omega \leftarrow \omega - \alpha * \nabla_w L_m(w)$$

}

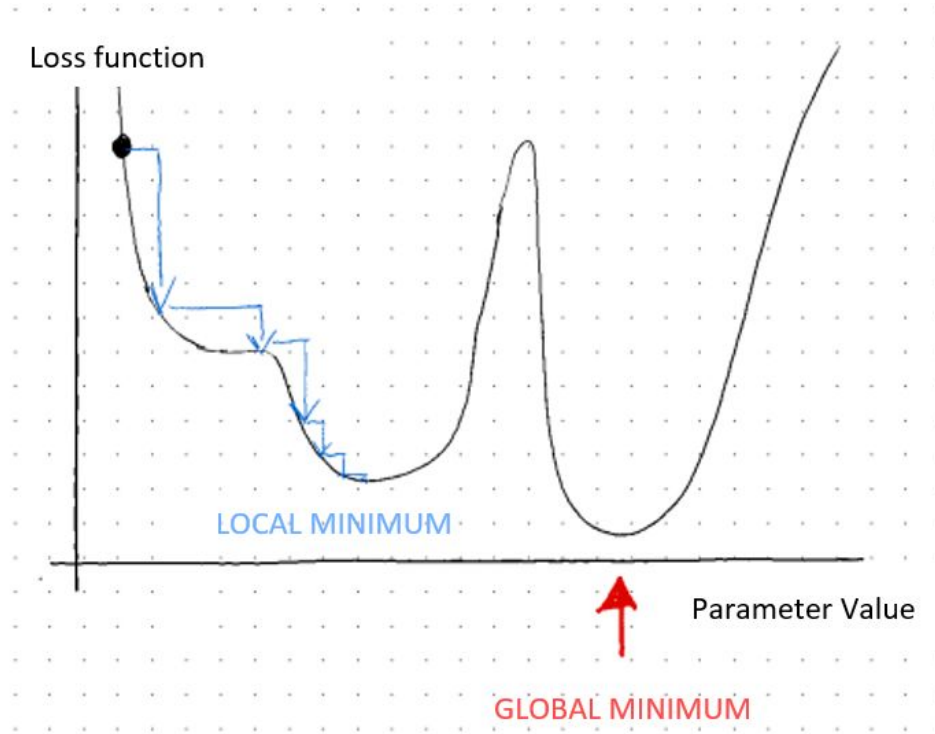
}



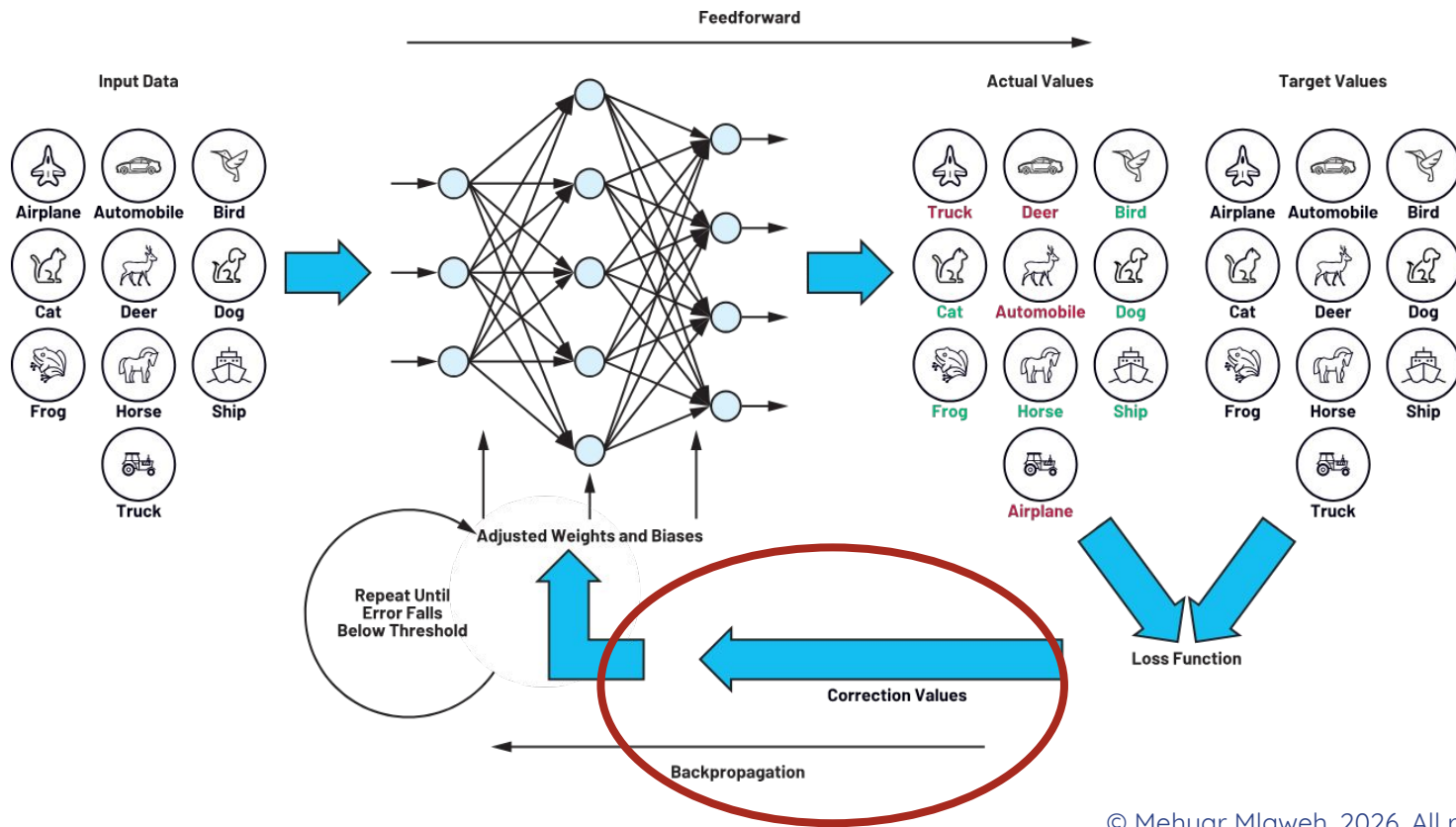
Learning rate (step size)

Training = repeatedly moving parameters in the direction that **reduces** loss

Gradient Descent:



Training a neural network



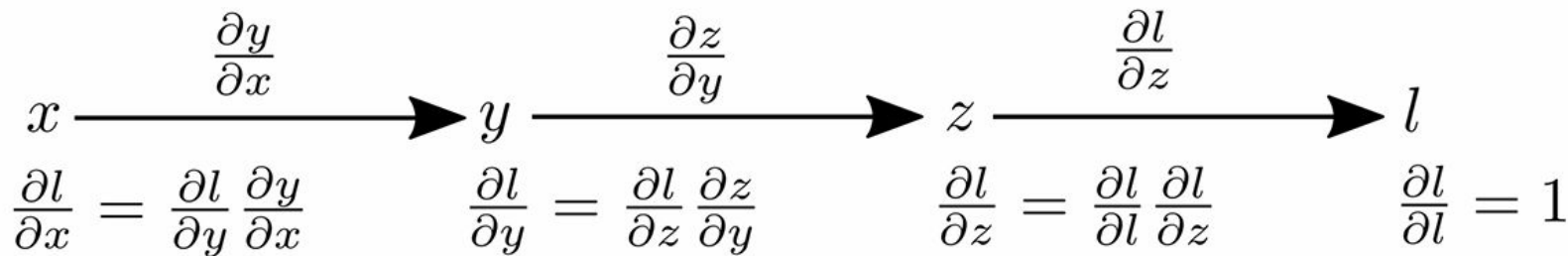
- **Gradient descent** updates parameters
- **Backpropagation** efficiently computes gradients for all parameters using the **Chain Rule**.
- It was first applied to NN in the eighties [Werbos, 1982, LeCun, 1985]

Chain Rule :

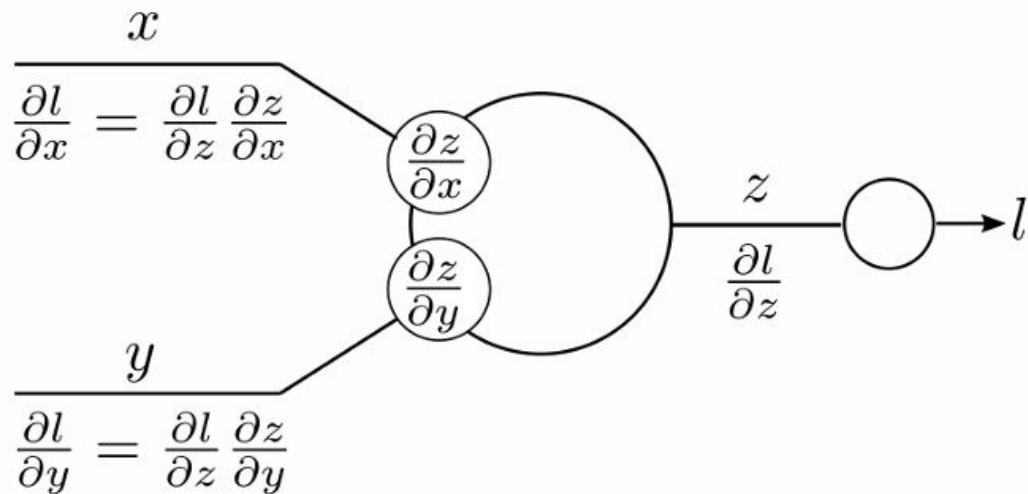
if $f(x)$ and $x(t)$ are functions, then

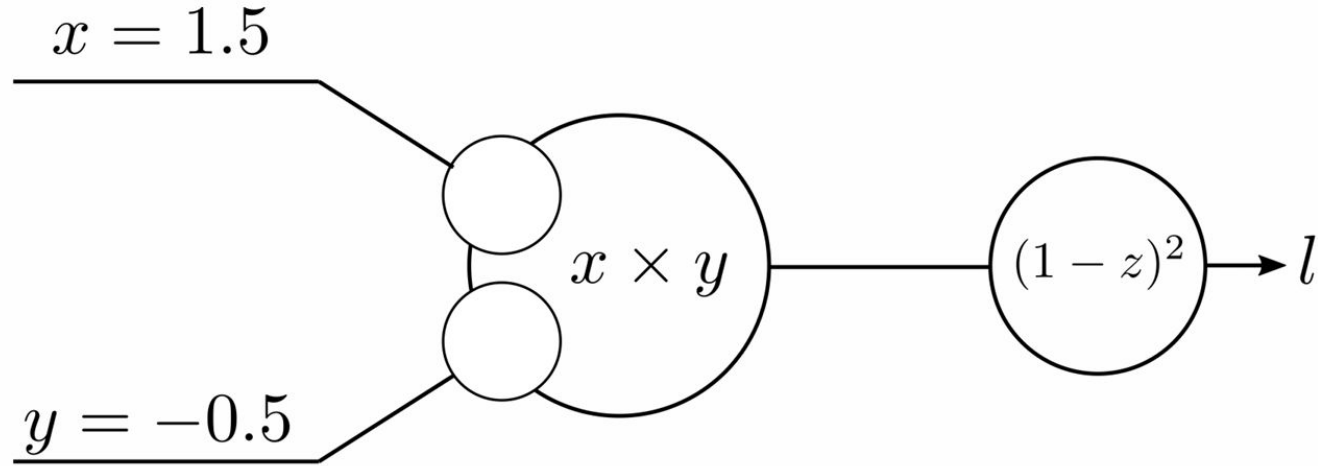
$$\frac{d}{dt} f(x(t)) = \frac{df}{dx} \frac{dx}{dt}.$$

Simple example



Another example



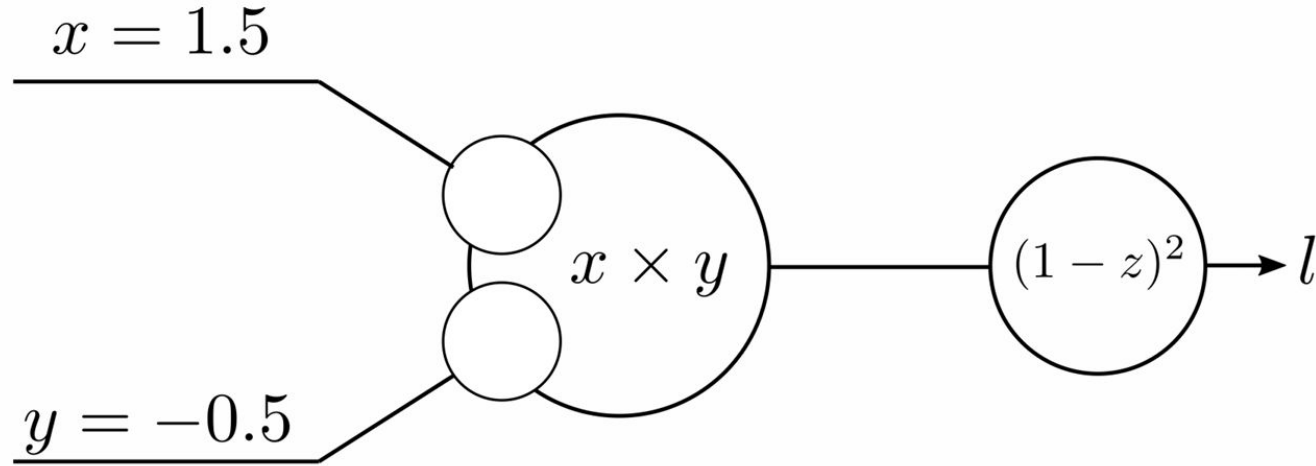


$\frac{\partial l}{\partial x}$ is equal to:

A/ 1.75

B/ -0.5

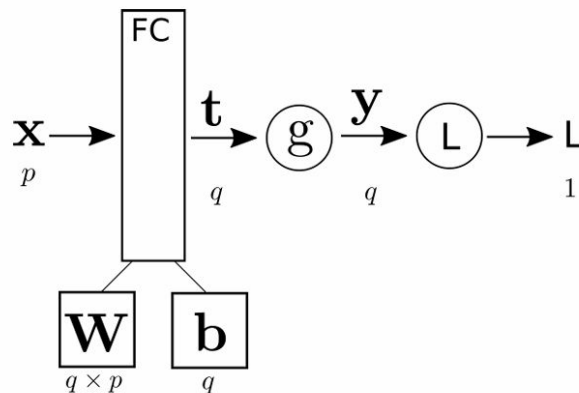
C/ 0.75



$\frac{\partial l}{\partial x}$ is equal to:

- A/ 1.75
- B/ -0.5
- C/ 0.75

General case



Setup:

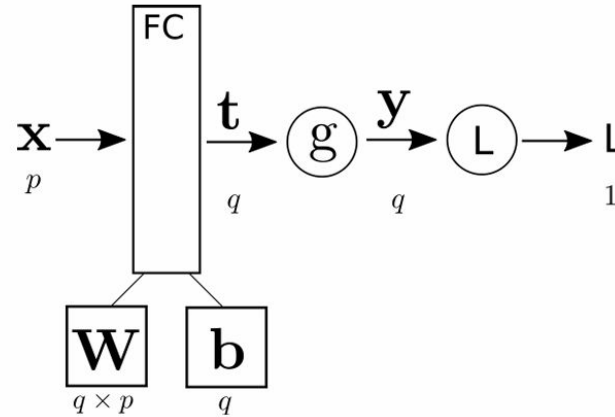
$$p, q \in \mathbb{N}^*$$

$$\mathbf{x} \in \mathbb{R}^p$$

$$\mathbf{W} \in \mathbb{R}^q \times \mathbb{R}^p$$

$$\mathbf{b}, \mathbf{t}, \mathbf{y} \in \mathbb{R}^q$$

$$L \in \mathbb{R}$$

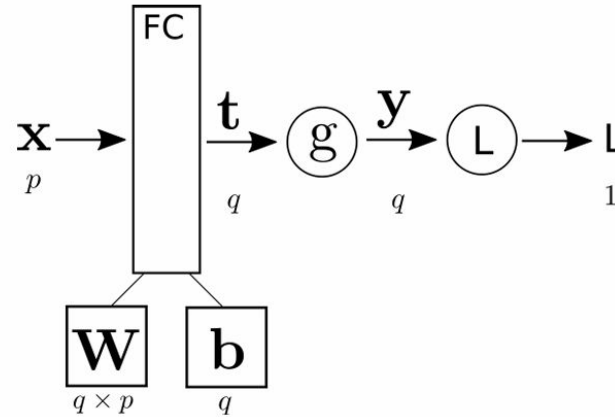


Forward pass:

$$\mathbf{t} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

$$\mathbf{y} = g(\mathbf{W}\mathbf{x} + \mathbf{b})$$

$$L = L(\mathbf{y})$$

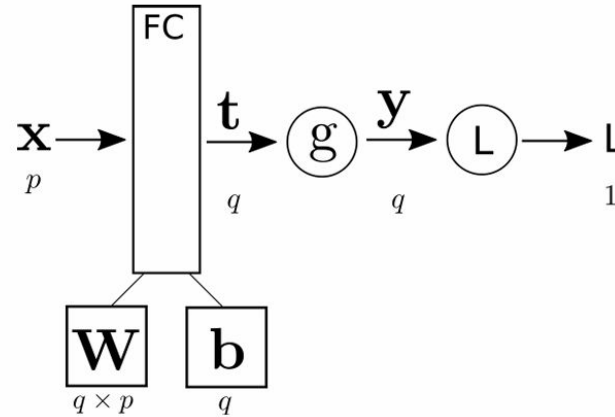


Local gradients:

Forward pass:

$$\begin{aligned}\mathbf{t} &= \mathbf{W}\mathbf{x} + \mathbf{b} \\ \mathbf{y} &= g(\mathbf{W}\mathbf{x} + \mathbf{b}) \\ L &= L(\mathbf{y})\end{aligned}$$

$$\begin{aligned}\frac{\partial \mathbf{t}}{\partial \mathbf{W}} &= \mathbf{x}^t \\ \frac{\partial \mathbf{t}}{\partial \mathbf{b}} &= Id_{(q)} \\ \frac{\partial \mathbf{y}}{\partial \mathbf{t}} &= g'(\mathbf{t})\end{aligned}$$

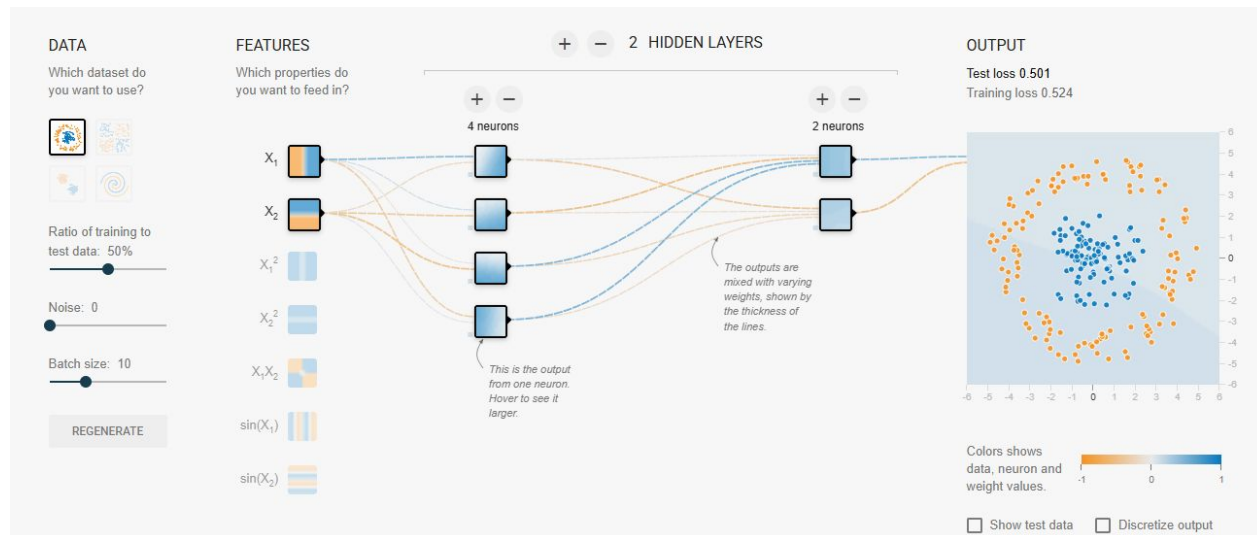


Backpropagation:

$$\begin{aligned}\frac{\partial L}{\partial \mathbf{W}} &= \frac{\partial L}{\partial \mathbf{t}} \cdot \frac{\partial \mathbf{t}}{\partial \mathbf{W}} \\ &= \frac{\partial L}{\partial \mathbf{y}} \odot \mathbf{g}'(\mathbf{t}) \cdot \mathbf{x}^t\end{aligned}$$

$$\begin{aligned}\frac{\partial L}{\partial \mathbf{b}} &= Id^t \cdot \frac{\partial L}{\partial \mathbf{t}} \\ &= \frac{\partial L}{\partial \mathbf{y}} \odot \mathbf{g}'(\mathbf{t})\end{aligned}$$

👉 Tensorflow Playground



Convolutional Neural Networks

Books

- Computer Vision: Models, Learning, and Inference (2012). <https://amzn.to/2rxrdOF>
- Programming Computer Vision with Python (2012). <https://amzn.to/2QKTaAL>
- Multiple View Geometry in Computer Vision (2004). <https://amzn.to/2LfHLE8>
- Computer Vision: Algorithms and Applications (2010). <https://amzn.to/2Lclt4J>
- Computer Vision: A Modern Approach (2002). <https://amzn.to/2rv7AqJ>
- Deep Learning for Computer Vision : Image Classification, Object Detection and Face Recognition in Python(2019). <https://machinelearningmastery.com/deep-learning-for-computer-vision/>
- Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow — Aurélien Géron (2019)
<https://amzn.to/3uZbN6Z>

Articles & Online Resources

- Krizhevsky, Sutskever, Hinton (2012) — ImageNet Classification with Deep CNNs (AlexNet — deep learning breakthrough)
- “Computer vision,” Wikipedia. https://en.wikipedia.org/wiki/Computer_vision
- “Machine vision,” Wikipedia. https://en.wikipedia.org/wiki/Machine_vision
- “Digital image processing,” Wikipedia. https://en.wikipedia.org/wiki/Digital_image_processing
- “Training Convolutional Neural Networks: What Is Machine Learning” Analog Devices.
<https://www.analog.com/en/resources/analog-dialogue/articles/training-convolutional-neural-networks-what-is-machine-learning-part-2.html>